

✓ Welcome to Colab!

Google Colab is available in VS Code!



Try the new [Google Colab extension](#) for Visual Studio Code. You can get up and running in just a few clicks:

- In VS Code, open the **Extensions** view and search for 'Google Colab' to install.
- Open the kernel selector by creating or opening any `.ipynb` notebook file in your local workspace and either running a cell or clicking the **Select Kernel** button in the top right.
- Click **Colab** and then select your desired runtime, sign in with your Google account, and you're all set!

See more details in our [announcement blog here](#).

✓ Access popular AI models via Google-Colab-AI Without an API Key

All users have access to most popular LLMs via the `google-colab-ai` Python library, and paid users have access to a wider selection of models. For more details, refer to the [getting started with google colab ai](#).

```
from google.colab import ai
response = ai.generate_text("What is the capital of France?")
```

✓ Explore the Gemini API

The Gemini API gives you access to Gemini models created by Google DeepMind. Gemini models are built from the ground up to be multimodal, so you can reason seamlessly across text, images, code, and audio.

How to get started?

- Go to [Google AI Studio](#) and log in with your Google account.
- [Create an API key](#).
- Use a quickstart for [Python](#), or call the REST API using [curl](#).

Discover Gemini's advanced capabilities

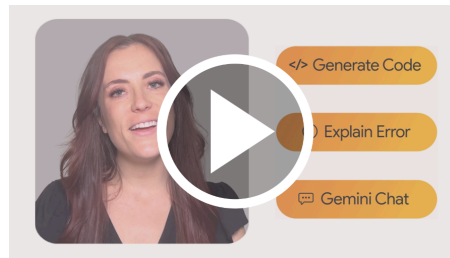
- Play with Gemini [multimodal outputs](#), mixing text and images in an iterative way.
- Discover the [multimodal Live API](#) (demo [here](#)).
- Learn how to [analyze images and detect items in your pictures](#) using Gemini (bonus, there's a [3D version](#) as well!).
- Unlock the power of [Gemini thinking model](#), capable of solving complex task with its inner thoughts.

Explore complex use cases

- Use [Gemini grounding capabilities](#) to create a report on a company based on what the model can find on internet.
- Extract [invoices and form data from PDF](#) in a structured way.
- Create [illustrations based on a whole book](#) using Gemini large context window and Imagen.

To learn more, check out the [Gemini cookbook](#) or visit the [Gemini API documentation](#).

Colab now has AI features powered by [Gemini](#). The video below provides information on how to use these features, whether you're new to Python, or a seasoned veteran.



What is Colab?

Colab, or "Colaboratory", allows you to write and execute Python in your browser, with

- Zero configuration required
- Access to GPUs free of charge
- Easy sharing

Whether you're a **student**, a **data scientist** or an **AI researcher**, Colab can make your work easier. Watch [Introduction to Colab](#) or [Colab Features You May Have Missed](#) to learn more, or just get started below!

Getting started

The document you are reading is not a static web page, but an interactive environment called a **Colab notebook** that lets you write and execute code.

For example, here is a **code cell** with a short Python script that computes a value, stores it in a variable, and prints the result:

```
seconds_in_a_day = 24 * 60 * 60
seconds_in_a_day
```

```
86400
```

To execute the code in the above cell, select it with a click and then either press the play button to the left of the code, or use the keyboard shortcut "Command/Ctrl+Enter". To edit the code, just click the cell and start editing.

Variables that you define in one cell can later be used in other cells:

```
seconds_in_a_week = 7 * seconds_in_a_day
seconds_in_a_week
```

```
604800
```

Colab notebooks allow you to combine **executable code** and **rich text** in a single document, along with **images**, **HTML**, **LaTeX** and more. When you create your own Colab notebooks, they are stored in your Google Drive account. You can easily share your Colab notebooks with co-workers or friends, allowing them to comment on your notebooks or even edit them. To learn more, see [Overview of Colab](#). To create a new Colab notebook you can use the File menu above, or use the following link: [create a new Colab notebook](#).

Colab notebooks are Jupyter notebooks that are hosted by Colab. To learn more about the Jupyter project, see [jupyter.org](#).

Data science

With Colab you can harness the full power of popular Python libraries to analyze and visualize data. The code cell below uses **numpy** to generate some random data, and uses **matplotlib** to visualize it. To edit the code, just click the cell and start editing.

You can import your own data into Colab notebooks from your Google Drive account, including from spreadsheets, as well as from Github and many other sources. To learn more about importing data, and how Colab can be used for data science, see the links below under [Working with Data](#).

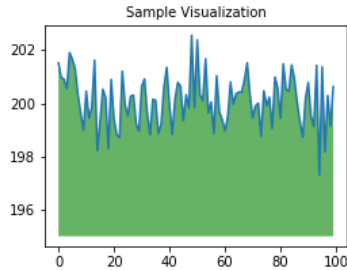
```
import numpy as np
import IPython.display as display
from matplotlib import pyplot as plt
import io
```

```
import base64

ys = 200 + np.random.randn(100)
x = [x for x in range(len(ys))]

fig = plt.figure(figsize=(4, 3), facecolor='w')
plt.plot(x, ys, '-')
plt.fill_between(x, ys, 195, where=(ys > 195), facecolor='g', alpha=0.6)
plt.title("Sample Visualization", fontsize=10)

data = io.BytesIO()
plt.savefig(data)
image = F"data:image/png;base64,{base64.b64encode(data.getvalue()).decode()}"
alt = "Sample Visualization"
display.display(display.Markdown(F"!!![{alt}]({image})"))
plt.close(fig)
```



Colab notebooks execute code on Google's cloud servers, meaning you can leverage the power of Google hardware, including [GPUs and TPUs](#), regardless of the power of your machine. All you need is a browser.

For example, if you find yourself waiting for **pandas** code to finish running and want to go faster, you can switch to a GPU Runtime and use libraries like [RAPIDS cuDF](#) that provide zero-code-change acceleration.

To learn more about accelerating pandas on Colab, see the [10 minute guide](#) or [US stock market data analysis demo](#).

Machine learning

With Colab you can import an image dataset, train an image classifier on it, and evaluate the model, all in just [a few lines of code](#).

Colab is used extensively in the machine learning community with applications including:

- Getting started with TensorFlow
- Developing and training neural networks
- Experimenting with TPUs
- Disseminating AI research
- Creating tutorials

To see sample Colab notebooks that demonstrate machine learning applications, see the [machine learning examples](#) below.

More Resources

Working with Notebooks in Colab

- [Overview of Colab](#)
- [Guide to Markdown](#)
- [Importing libraries and installing dependencies](#)
- [Saving and loading notebooks in GitHub](#)
- [Interactive forms](#)
- [Interactive widgets](#)

Working with Data

- [Loading data: Drive, Sheets, and Google Cloud Storage](#)

- [Charts: visualizing data](#)
- [Getting started with BigQuery](#)

Machine Learning

These are a few of the notebooks related to Machine Learning, including Google's online Machine Learning course. See the [full course website](#) for more.

- [Intro to Pandas DataFrame](#)
- [Intro to RAPIDS cuDF to accelerate pandas](#)
- [Getting Started with cuML's accelerator mode](#)

Using Accelerated Hardware

- [Train a CNN to classify handwritten digits on the MNIST dataset using Flax NNX API](#)
- [Train a Vision Transformer \(ViT\) for image classification with JAX](#)
- [Text classification with a transformer language model using JAX](#)

Featured examples

- [Train a miniGPT language model with JAX AI Stack](#)
- [LoRA/QLoRA finetuning for LLM using Tux](#)
- [Parameter-efficient fine-tuning of Gemma with LoRA and QLoRA](#)
- [Loading Hugging Face Transformers Checkpoints](#)
- [8-bit Integer Quantization in Keras](#)
- [Float8 training and inference with a simple Transformer model](#)
- [Pretraining a Transformer from scratch with KerasHub](#)
- [Simple MNIST convnet](#)
- [Image classification from scratch using Keras 3](#)
- [Image Classification with KerasHub](#)

```
import os
from google.colab import userdata

import os
try:
    os.environ["KGAT_8e0620c18d0fff6221c87775db272e25"] = userdata.get("KGAT_8e0620c18d0fff6221c87775db272e25")
except userdata.SecretNotFoundError:
    print("Warning: Secret 'KGAT_8e0620c18d0fff6221c87775db272e25' not found. Please add it to Colab Secrets if you intend to
```

Warning: Secret 'KGAT_8e0620c18d0fff6221c87775db272e25' not found. Please add it to Colab Secrets if you intend to use it.

```
!pip install -q kaggle
```

```
!kaggle datasets list
```

You must authenticate before you can call the Kaggle API.
Follow the instructions to authenticate at: <https://github.com/Kaggle/kaggle-cli/blob/main/docs/README.md#authentication>

```
import os
os.environ["KAGGLE_API_TOKEN"] = "KGAT_beaf6fd5ca2ac5baa0a8660ea821a7b1"
```

```
!pip install -q kaggle
```

```
!kaggle datasets list
```

```
Warning: Looks like you're using an outdated `kaggle` version (installed: 2.0.2), please consider upgrading to the latest vers
ref
-----
rauffauzanrambe/fifa-world-cup-2026-player-performance-dataset  FIFA World Cup 2026 Player Performance Dataset
ashyou09/apple-products-pricing-dataset-2020-2026             Apple Products Pricing Dataset (2020-2026) 🍏
abbas829/ecommerce-sales-dataset                               Ecommerce sales dataset
uditjain13/social-media-screen-time-and-mental-health-2026    Social Media, Screen Time & Mental Health 2026
arjunmehta1992/chocolate-sales-in-2022-2023                  Chocolate Sales in 2022-2023
mirzayasirabdullah07/netflix-dataset                           Netflix Dataset
sunil123kumar/social-media-impact-on-mental-health            Social_Media_Impact_on_Mental Health
uditjain13/nvidia-gpu-sales-synthetic-2026                    Nvidia GPU Sales Data 2026
```

lavesadjadon/ai-impact-on-students	Impact of Ai on Students
dreamtensor/gaming-addiction-and-mental-health-analysis	Gaming Addiction and Mental Health Analysis
zynicide/wine-reviews	Wine Reviews
datasnaek/chess	Chess Game Dataset (Lichess)
residentmario/ramen-ratings	Ramen Ratings
dansbecker/powerlifting-database	powerlifting-database
jpmiller/publicassistance	US Public Food Assistance 1 - WIC
datasnaek/youtube-new	Trending YouTube Video Statistics
kaggle/pokemon-tcg-ai-battle-episodes-2026-07-07	The Pokémon Company - PTCG AI Battle Challenge Simulation Epis
nolanbconaway/pitchfork-data	18,393 Pitchfork Reviews
rtatman/188-million-us-wildfires	1.88 Million US Wildfires
nasa/kepler-exoplanet-search-results	Kepler Exoplanet Search Results

```
!kaggle datasets list -s "cats vs dogs"
```

Warning: Looks like you're using an outdated `kaggle` version (installed: 2.0.2), please consider upgrading to the latest vers

ref	title	size	lastUpdated
shaunthesheep/microsoft-catsvsdogs-dataset	Cats-vs-Dogs	825979578	2020-03-12 0
biaiscience/dogs-vs-cats	Dogs vs Cats	856265363	2017-12-05 1
pybear/cats-vs-dogs	Cats -- VS -- Dogs	825979578	2020-10-18 1
sujoykapadnis/cats-vs-dogs	Cats Vs Dogs	1926	2023-09-25 0
moazeldsokyx/dogs-vs-cats	dogs vs cats	856210622	2023-08-07 0
princely84/dogsvscats	Dogs Vs Cats	571120858	2025-09-23 1
dhirensk/cats-vs-dogs-training8000test2000	Cats vs Dogs Training8000Test2000	457145384	2018-11-21 0
salader/dogsvscats	dogsVScats	1142791694	2018-11-30 1
alvarole/asirra-cats-vs-dogs-object-detection-dataset	Asirra: Cats vs Dogs Object Detection Dataset	25404614	2020-07-23 0
blourdthuraju/dogs-vs-cats-classification	dogs-vs-cats-classification	812278276	2026-02-24 0
yashdogra/cats-and-dogs	Cats Vs Dogs	792627373	2025-04-01 1
arnaudeq/cats-vs-dogs-1000	Cats Vs Dogs (1000)	185348382	2018-11-22 2
abhinavnayak/catsvsdogs-transformed	Cats vs Dogs - 2000 images (224x224)	17380278	2021-12-11 1
hassanaitnacer/dogs-vs-cats	Dogs vs Cats	227695717	2022-12-28 2
pocahontas1010/dogs-vs-cats-for-pytorch	Dogs vs Cats for Pytorch	1144101694	2018-12-03 2
sarimr/dogs-vs-cats-dataset	Dogs Vs Cats Dataset	857115363	2024-10-12 1
pushpakhinglaspure/cats-vs-dogs	cats vs dogs	572950024	2024-01-31 0
karakaggle/kaggle-cat-vs-dog-dataset	Kaggle: Cat VS Dog Dataset	825666877	2020-05-01 1
kanwalinder/cats-vs-dogs-redux-transfer-features	Cats vs Dogs Redux Transfer Features	1345261572	2018-08-22 0
kunalgupta2616/dog-vs-cat-images-data	Dog vs Cat Images Data	1038798343	2020-09-01 0

```
!kaggle datasets list -s "gold"
```

Warning: Looks like you're using an outdated `kaggle` version (installed: 2.0.2), please consider upgrading to the latest vers

ref	title	size	lastU
tunguz/gold-prices	Gold Prices	5555	2021-
sid321axn/gold-price-prediction-dataset	Gold Price Prediction Dataset	379629	2021-
krupalpatel07/gold-price-dynamics	Gold Price Dynamics	85982	2026-
franciscogcc/financial-data	Gold Price Regression	522571	2024-
sahilwagh/gold-stock-prices	Gold Stock Prices	94742	2024-
nward7/gold-historical-datasets	Gold Historical Datasets	105710	2021-
odins0n/monthly-gold-prices	Monthly Gold Prices (1979-2021)	55389	2021-
rifatalam3/gold-price-trends	Gold Price Trends	85994	2026-
novandraanugrah/xauusd-gold-price-historical-data-2004-2024	XAU/USD Gold Price Historical Data (2004-2026)	103449070	2026-
hemil26/gold-rates-1985-jan-2022	Gold rates (1985 - Present)	219022	2023-
cvergnolle/gold-price-and-relevant-metrics	Gold Price & Relevant Metrics	36922	2024-
willianoliveiragibin/gold-coin-prices	Gold Coin Prices	61303	2024-
gvyshnya/gold-future-prices	Gold Future Prices	50847	2024-
unanimad/golden-globe-awards	Golden Globe Awards, 1944 - 2020	119903	2020-
vishardmehta/gold-price-forecasting-dataset	Gold Price Forecasting Dataset	138149	2026-
uzmaakhtar/gold-price-history-18332026	Gold Price History (1833-2026)	144506	2026-
altruistdelhite04/gold-price-data	Gold Price Data	42904	2019-
bhanupratapbiswas/gold-prices	Gold Prices 📈📉📊	2539	2
kapturovalexander/gold-and-silver-prices-2013-2023	🏆🥈 Gold and Silver prices (2013-2025)	117371	20
umerhaddii/gold-stock-data-2025	GOLD Stock Data 2025	102564	2025-

```
!kaggle datasets download -d shaunthesheep/microsoft-catsvsdogs-dataset -p /content/data
```

Warning: Looks like you're using an outdated `kaggle` version (installed: 2.0.2), please consider upgrading to the latest vers

Dataset URL: <https://www.kaggle.com/datasets/shaunthesheep/microsoft-catsvsdogs-dataset>

License(s): other

Downloading microsoft-catsvsdogs-dataset.zip to /content/data

100% 788M/788M [00:08<00:00, 100MB/s]

```
import zipfile
import os
```

```
zip_path = "/content/data/microsoft-catsvsdogs-dataset.zip"
extract_path = "/content/data"

with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall(extract_path)

print("Dataset extracted successfully.")
```

Dataset extracted successfully.

```
import os

base_dir = "/content/data/PetImages"

print(os.listdir(base_dir))
print("Cat images:", len(os.listdir(os.path.join(base_dir, "Cat"))))
print("Dog images:", len(os.listdir(os.path.join(base_dir, "Dog"))))
```

```
['Dog', 'Cat']
Cat images: 12501
Dog images: 12501
```

```
from PIL import Image
import os

classes = ["Cat", "Dog"]

removed = 0

for class_name in classes:
    folder_path = os.path.join(base_dir, class_name)

    for img_name in os.listdir(folder_path):
        img_path = os.path.join(folder_path, img_name)

        try:
            img = Image.open(img_path)
            img.verify()
        except:
            os.remove(img_path)
            print(class_name)
            removed += 1

print("Corrupted images removed:", removed)
```

```
Cat
Cat
/usr/local/lib/python3.12/dist-packages/PIL/TiffImagePlugin.py:950: UserWarning: Truncated File Read
  warnings.warn(str(msg))
Dog
Dog
Corrupted images removed: 4
```

```
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator

IMG_SIZE = 150
BATCH_SIZE = 32

datagen = ImageDataGenerator(
    rescale=1./255,
    validation_split=0.2
)

train_data = datagen.flow_from_directory(
    base_dir,
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    class_mode="binary",
    subset="training"
)

val_data = datagen.flow_from_directory(
    base_dir,
    target_size=(IMG_SIZE, IMG_SIZE),
```

```

    batch_size=BATCH_SIZE,
    class_mode="binary",
    subset="validation"
)

```

Found 20000 images belonging to 2 classes.
Found 4998 images belonging to 2 classes.

```
print(train_data.class_indices)
```

```
{'Cat': 0, 'Dog': 1}
```

```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout

model = Sequential([
    Conv2D(32, (3, 3), activation="relu", input_shape=(IMG_SIZE, IMG_SIZE, 3)),MaxPooling2D(2, 2),

    Conv2D(64, (3, 3), activation="relu"),MaxPooling2D(2, 2),

    Conv2D(128, (3, 3), activation="relu"),MaxPooling2D(2, 2),

    Flatten(),

    Dense(128, activation="relu"),

    Dense(64, activation="relu"),

    Dense(1, activation="sigmoid")
])

model.summary()

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input\_shape` argument to `Conv2D` layers. It will be ignored. In the future, this will result in an error.
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 148, 148, 32)	896
max_pooling2d (MaxPooling2D)	(None, 74, 74, 32)	0
conv2d_1 (Conv2D)	(None, 72, 72, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 36, 36, 64)	0
conv2d_2 (Conv2D)	(None, 34, 34, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 17, 17, 128)	0
flatten (Flatten)	(None, 36992)	0
dense (Dense)	(None, 128)	4,735,104
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 1)	65

Total params: 4,836,673 (18.45 MB)

Trainable params: 4,836,673 (18.45 MB)

Non-trainable params: 0 (0.00 MB)

```

model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

```

```

history = model.fit(
    train_data,
    validation_data=val_data,
    epochs=20
)

```

```

Epoch 1/20
526/625 ----- 2:33 2s/step - accuracy: 0.6126 - loss: 0.6422/usr/local/lib/python3.12/dist-packages/PIL/TiffIma
warnings.warn(str(msg))
625/625 ----- 1058s 2s/step - accuracy: 0.6886 - loss: 0.5773 - val_accuracy: 0.7625 - val_loss: 0.4881
Epoch 2/20
625/625 ----- 1063s 2s/step - accuracy: 0.7824 - loss: 0.4603 - val_accuracy: 0.8081 - val_loss: 0.4148
Epoch 3/20
625/625 ----- 1052s 2s/step - accuracy: 0.8296 - loss: 0.3761 - val_accuracy: 0.8191 - val_loss: 0.3903
Epoch 4/20
625/625 ----- 1040s 2s/step - accuracy: 0.8735 - loss: 0.2908 - val_accuracy: 0.7995 - val_loss: 0.4340
Epoch 5/20
625/625 ----- 1037s 2s/step - accuracy: 0.9200 - loss: 0.1961 - val_accuracy: 0.8257 - val_loss: 0.4545
Epoch 6/20
625/625 ----- 1033s 2s/step - accuracy: 0.9584 - loss: 0.1107 - val_accuracy: 0.8219 - val_loss: 0.5571
Epoch 7/20
625/625 ----- 1036s 2s/step - accuracy: 0.9758 - loss: 0.0663 - val_accuracy: 0.8107 - val_loss: 0.6926
Epoch 8/20
625/625 ----- 1037s 2s/step - accuracy: 0.9847 - loss: 0.0438 - val_accuracy: 0.8273 - val_loss: 0.8748
Epoch 9/20
625/625 ----- 1031s 2s/step - accuracy: 0.9895 - loss: 0.0321 - val_accuracy: 0.8141 - val_loss: 1.0800
Epoch 10/20
625/625 ----- 1035s 2s/step - accuracy: 0.9912 - loss: 0.0277 - val_accuracy: 0.8187 - val_loss: 1.0155
Epoch 11/20
625/625 ----- 1024s 2s/step - accuracy: 0.9903 - loss: 0.0284 - val_accuracy: 0.8083 - val_loss: 1.1468
Epoch 12/20
625/625 ----- 1025s 2s/step - accuracy: 0.9887 - loss: 0.0323 - val_accuracy: 0.8267 - val_loss: 1.0834
Epoch 13/20
625/625 ----- 1032s 2s/step - accuracy: 0.9948 - loss: 0.0186 - val_accuracy: 0.8129 - val_loss: 1.0154
Epoch 14/20
625/625 ----- 1040s 2s/step - accuracy: 0.9969 - loss: 0.0108 - val_accuracy: 0.8147 - val_loss: 1.2047
Epoch 15/20
625/625 ----- 1023s 2s/step - accuracy: 0.9900 - loss: 0.0316 - val_accuracy: 0.8017 - val_loss: 1.1073
Epoch 16/20
625/625 ----- 1026s 2s/step - accuracy: 0.9924 - loss: 0.0233 - val_accuracy: 0.8153 - val_loss: 1.0710
Epoch 17/20
625/625 ----- 1027s 2s/step - accuracy: 0.9952 - loss: 0.0148 - val_accuracy: 0.8211 - val_loss: 1.0555
Epoch 18/20
625/625 ----- 1025s 2s/step - accuracy: 0.9951 - loss: 0.0156 - val_accuracy: 0.8241 - val_loss: 1.0546
Epoch 19/20
625/625 ----- 1027s 2s/step - accuracy: 0.9982 - loss: 0.0058 - val_accuracy: 0.8229 - val_loss: 1.3420
Epoch 20/20
625/625 ----- 1047s 2s/step - accuracy: 0.9941 - loss: 0.0188 - val_accuracy: 0.8183 - val_loss: 1.3548

```

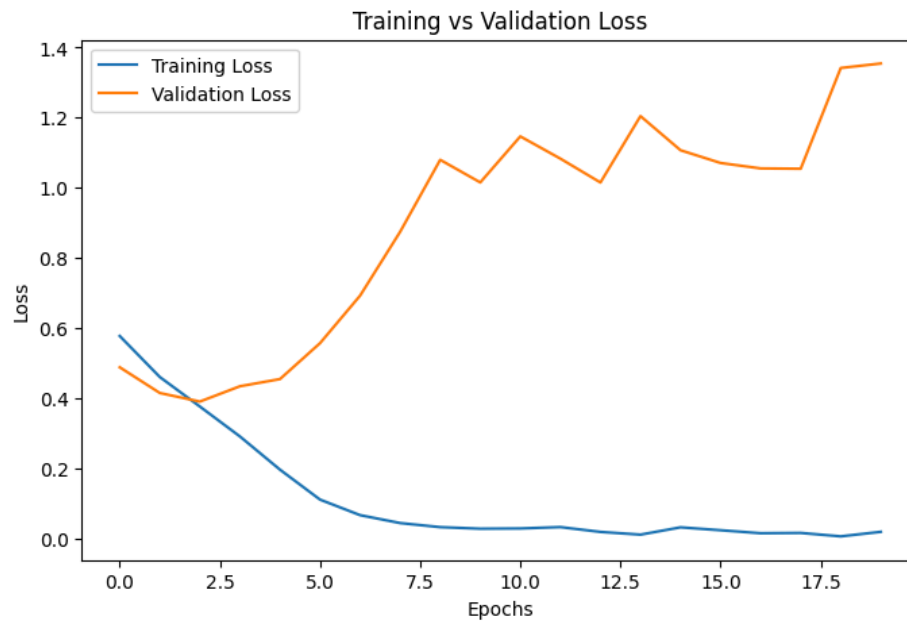
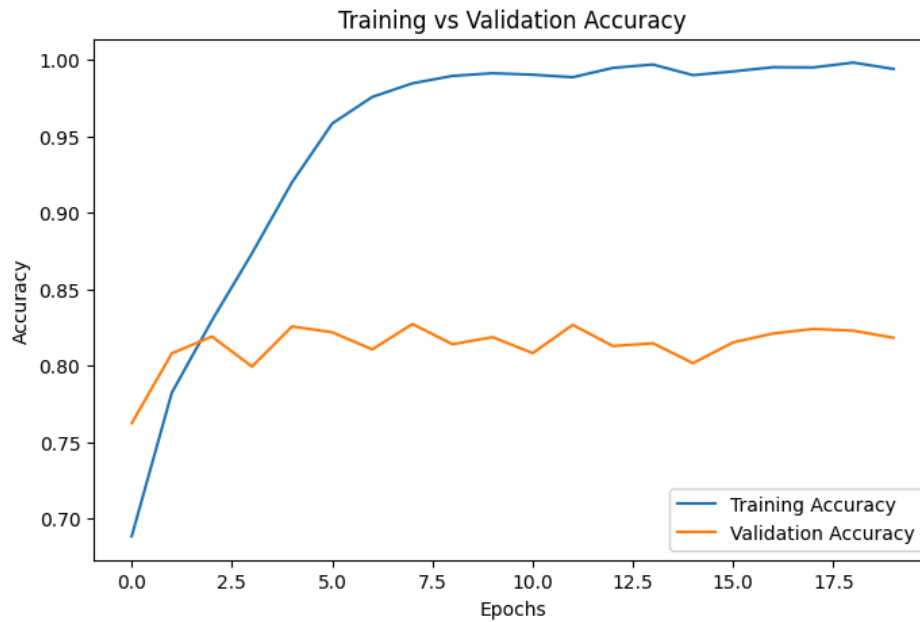
```

import matplotlib.pyplot as plt

plt.figure(figsize=(8, 5))
plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.title("Training vs Validation Accuracy")
plt.legend()
plt.show()

plt.figure(figsize=(8, 5))
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.title("Training vs Validation Loss")
plt.legend()
plt.show()

```

```
loss, accuracy = model.evaluate(val_data)
```

```
print("Validation Loss:", loss)
```

```
print("Validation Accuracy:", accuracy)
```

157/157 ————— 75s 475ms/step - accuracy: 0.8183 - loss: 1.3548

Validation Loss: 1.3547831773757935

Validation Accuracy: 0.8183273077011108

```
model.save("/content/cat_dog_basic_cnn_model.h5")
```

```
print("Model saved successfully.")
```

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is deprecated. Model saved successfully.

```
from google.colab import files
from tensorflow.keras.preprocessing import image
import numpy as np
import matplotlib.pyplot as plt

uploaded = files.upload()

for file_name in uploaded.keys():
```

```

...img_path = file_name

...img = image.load_img(img_path, target_size=(IMG_SIZE, IMG_SIZE))
...img_array = image.img_to_array(img)
...img_array = img_array / 255.0
...img_array = np.expand_dims(img_array, axis=0)

...prediction = model.predict(img_array)

...plt.imshow(img)
...plt.axis("off")
...plt.show()

...if prediction[0][0] > 0.5:
...    print("Prediction: Dog")
...    print("Confidence:", prediction[0][0])
...else:
...    print("Prediction: Cat")
...    print("Confidence:", 1 - prediction[0][0])

```

Choose Files Picture .jpg

Picture .jpg(image/jpeg) - 30896 bytes, last modified: 7/12/2026 - 100% done

Saving Picture .jpg to Picture .jpg

1/1  0s 281ms/step



Prediction: Cat
Confidence: 0.78363806

```

from google.colab import files
from tensorflow.keras.preprocessing import image
import numpy as np
import matplotlib.pyplot as plt

uploaded = files.upload()

for file_name in uploaded.keys():
    img_path = file_name

    img = image.load_img(img_path, target_size=(IMG_SIZE, IMG_SIZE))
    img_array = image.img_to_array(img)
    img_array = img_array / 255.0
    img_array = np.expand_dims(img_array, axis=0)

    prediction = model.predict(img_array)

    plt.imshow(img)
    plt.axis("off")
    plt.show()

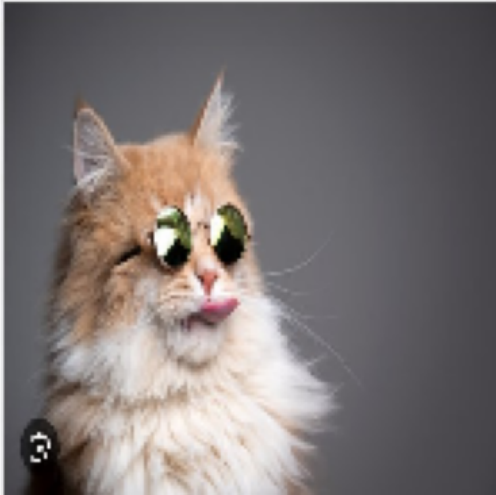
    if prediction[0][0] > 0.5:
        print("Prediction: Dog")
        print("Confidence:", prediction[0][0])
    else:
        print("Prediction: Cat")
        print("Confidence:", 1 - prediction[0][0])

```

[Choose Files](#) Photo1.jpg**Photo1.jpg**(image/jpeg) - 23236 bytes, last modified: 7/12/2026 - 100% done

Saving Photo1.jpg to Photo1.jpg

1/1 ————— 0s 58ms/step



Prediction: Dog

Confidence: 0.99998915

```
from google.colab import files
from tensorflow.keras.preprocessing import image
import numpy as np
import matplotlib.pyplot as plt

uploaded = files.upload()

for file_name in uploaded.keys():
    img_path = file_name

    img = image.load_img(img_path, target_size=(IMG_SIZE, IMG_SIZE))
    img_array = image.img_to_array(img)
    img_array = img_array / 255.0
    img_array = np.expand_dims(img_array, axis=0)

    prediction = model.predict(img_array)

    plt.imshow(img)
    plt.axis("off")
    plt.show()

    if prediction[0][0] > 0.5:
        print("Prediction: Dog")
        print("Confidence:", prediction[0][0])
    else:
        print("Prediction: Cat")
        print("Confidence:", 1 - prediction[0][0])
```

[Choose Files](#) snap.jpg**snap.jpg**(image/jpeg) - 27271 bytes, last modified: 7/12/2026 - 100% done

Saving snap.jpg to snap.jpg

1/1 ————— 0s 63ms/step



